Sentiment Analysis of Code-Mixed Telugu and English Reviews Telugu Film Reviews and Comments

CM701PC Deep Learning and CM751PC Deep Learning Lab – Complex Engineering Problem Task

Task 2 · Variant 2B · Team 6

B.Tech CSE (AI&ML), VII Semester, Regulation MR-22

Department of Emerging Technologies, Mahatma Gandhi Institute of Technology (Autonomous), Hyderabad

Academic Year: 2025–26

Team members and contributions

Roll number	Name	Allotted role	Contribution
23261A6639	Parsi Sriram	Data lead	Data generation and cleaning, annotation guide, data card, train/validation/test splitting, error analysis
23261A6654	T. Sathwik	Model lead	Building and training SimpleRNN, LSTM and GRU models, reproducible notebook code and README
23261A6609	Ch. Rohan	Evaluation and report lead	Metrics and plots, checking results against every target, answers to report questions, compiling the report

1. PROBLEM UNDERSTANDING

Task. Classify Telugu film reviews and comments, written in Romanized Telugu mixed with English, as Positive, Negative or Mixed (at least one clearly positive and one clearly negative opinion), e.g. “Story excellent but screenplay chala slow ga undi.” Sentiment analysis in general is surveyed in [5]; Telugu–English code-mixed text with noisy spelling has been handled by normalisation [9].

Constraints and trade-offs identified at the start. (a) Romanized Telugu has no standard spelling (bagundi/baagundi/bagundhi), so unseen spellings are expected. (b) Reviewers open politely and complain last, so truncating a long review can delete the deciding sentence; a longer maximum length keeps it but costs CPU time. (c) Gated and bidirectional layers capture more context but predict more slowly. (d) Spelling robustness (augmentation, normalisation) may cost standard accuracy. (e) English IMDB pretraining may add sentiment knowledge but shares little vocabulary with Romanized Telugu. Targets: Macro F1 ≥ 0.65, Mixed F1 ≥ 0.45, 10,000 reviews on CPU in < 60 s, and a defended maximum length.

Scope. No real, human-labelled Romanized Telugu corpus was available. All experiments use a synthetic, rule-labelled corpus, so results show that the pipeline works on this corpus and do not establish performance on real film reviews.

2. DATA PREPARATION AND EXPLORATORY ANALYSIS

Source and annotation. The 6,000 labelled reviews (2,000 per class) were generated by a clause-template grammar (aspect + evaluative phrase) in Romanized Telugu with English words, with injected spelling variants, negation, sarcasm, hedged-ambiguous Mixed reviews, and long reviews (filler between clauses) in every class. They are synthetic, not self-collected, contain no Telugu-script characters, and are labelled by the generation rule following the written annotation guide; no human annotated them. Missing values: 0; duplicate texts: 0. For unseen-data testing, a validation set (600) and test set (900) were generated so that every review contains at least one token absent from training (new spellings, slang or English words, aspects) from different halves of a novel lexicon not reviewed by a native speaker. A pool of 10,000 distinct reviews was used only for CPU timing.

Table 1. Class balance, split sizes and review statistics (synthetic corpus)

Class	Train	Val	Test	Mean tokens	Max tokens	Negation %	Spelling var. %	Long-form %
Positive	1407	340	253	31.1	123	28	66	34
Negative	1452	248	300	32.2	131	51	84	34
Mixed	1415	309	276	31.0	116	37	81	39
Total	4274	897	829	31.4	131	39	77	36

Preprocessing and leakage control. Text is lower-cased and Unicode-normalised; punctuation becomes separate tokens; Romanized Telugu words and negations (ledu, kaadu, nachaledu, baaledu) are kept and verified to survive cleaning. A Keras tokenizer with an OOV token is fitted on training reviews only; sequences are post-padded and post-truncated. Splits are group-aware (group = label, pattern, primary aspect; 70/15/15 % per class) with no group shared across splits, and the seed is fixed. Clause templates are still shared across groups, and the standard test set contains no unseen token, which is why the unseen-data sets were added. Exploration shows balanced classes and long reviews in every class (long-form share 34% Positive, 34% Negative, 39% Mixed), so length alone does not reveal the label; the longest review has 131 tokens.

3. MODELS AND EXPERIMENTS

Architecture. Embedding(48, mask_zero) → one recurrent layer (64 units, dropout 0.2) → Dense(24, ReLU) → Dropout(0.2) → Dense(3, Softmax), P(k | review) = exp(z_k)/Σ_j exp(z_j); Adam [8] (1e-3), batch 32, cross-entropy, ≤ 12 epochs, early stopping on validation loss (patience 2), seeds 42–44. Only the recurrent layer changes: SimpleRNN, LSTM or GRU. The unit count, embedding size, dropout and learning rate were fixed, not tuned, and bidirectional layers were not tested; these choices therefore lack an experiment.

Training strategies. A: random initialisation, plain text. A2: scratch plus noise augmentation (random character edits on 20% of tokens in 70% of reviews, 4% token dropout) and inference-time normalisation mapping an unseen token to the nearest training word when string similarity ≥ 0.8. B: English IMDB initialisation [6], designed to pre-train each architecture on IMDB, copy embedding rows by matching word strings and all recurrent weights, reset the dense and output layers, and fine-tune.

Strategy B executed. Source: Pang & Lee polarity corpus v2.0 (IMDb-sourced; 1,600 train / 400 test). The Keras IMDB set (Maas et al. [6]) could not be downloaded in the execution environment, so the IMDb-sourced Pang and Lee polarity corpus [11] was used instead; it is far smaller (2,000 reviews). Each architecture was first trained as a binary English sentiment classifier (6 epochs, input length 256; source test accuracy SimpleRNN 0.468, LSTM 0.540, GRU 0.627). Transferred: the embedding rows of the 85 of 171 target tokens (50%) that also occur in the source vocabulary, matched by word string (all other rows stay random), plus all recurrent kernel, recurrent-kernel and bias weights. Re-initialised: Dense(24) and the 3-class softmax output. All layers were fine-tuned with the same optimiser, schedule and early stopping as strategy A.

Table 2. Maximum sequence length study (validation Macro F1, seed 42; baseline models)

Max length	Truncated (%)	Final complaint kept (%)	RNN	LSTM	GRU	Mean
64	21.1	51.0	0.903	0.954	0.938	0.932
128	1.8	99.3	0.978	0.999	1.000	0.992
192	0.0	100.0	0.974	0.995	0.955	0.975
256	0.0	100.0	0.974	0.995	0.955	0.975

Where the line was drawn: L = 128, the highest mean validation Macro F1 (rule fixed beforehand; ties to the shorter length). At L = 64, 21.1% of training reviews are truncated and only 51.0% of complaint-last reviews keep their complaint, and every architecture is worst there. L = 192 and 256 truncate nothing yet scored lower than 128; the cause is unexplained and may be seed noise. The sweep predates the robustness changes and CPU time was not measured per length.

Table 3. Three recurrent layers × training strategies; mean (± SD) over 3 seeds. Left: standard test (n = 829); right: unseen-data test (n = 900).

Recurrent layer (strategy)	Acc.	Macro F1	Mixed F1	Kappa	Unseen Macro F1	Unseen Mixed F1
SimpleRNN (A scratch)	0.9855	0.9855 ± 0.0122	0.9916	0.9782	0.6071 ± 0.0193	0.5936
SimpleRNN (A2 scratch + robust)	0.9759	0.9763 ± 0.0092	0.9727	0.9637	0.7503 ± 0.0358	0.7753
SimpleRNN (B English transfer)	0.9851	0.9854 ± 0.0133	0.9881	0.9776	0.6100 ± 0.0220	0.5851
LSTM (A scratch)	0.9988	0.9988 ± 0.0012	0.9994	0.9982	0.6057 ± 0.0172	0.5677
LSTM (A2 scratch + robust)	0.9972	0.9972 ± 0.0030	0.9982	0.9958	0.8052 ± 0.0092	0.8470
LSTM (B English transfer)	0.9996	0.9996 ± 0.0007	1.0000	0.9994	0.5989 ± 0.0145	0.5800
GRU (A scratch)	0.9964	0.9964 ± 0.0012	1.0000	0.9946	0.6208 ± 0.0166	0.5811
GRU (A2 scratch + robust)	0.9497	0.9493 ± 0.0807	0.9905	0.9249	0.7310 ± 0.0437	0.8091
GRU (B English transfer)	0.9996	0.9996 ± 0.0007	1.0000	0.9994	0.6077 ± 0.0170	0.5779

Model selection used validation data only: the mean of in-distribution and unseen-data validation Macro F1, averaged over seeds, then the best seed of the winning configuration. Neither test set influenced any choice.

4. RESULTS AGAINST EVERY PERFORMANCE TARGET

Table 4. Results against targets (selected model)

Requirement	Measured	Status
Macro F1 on test set ≥ 0.65	0.9976	PASS
Mixed-class F1 ≥ 0.45	0.9982	PASS
10,000 reviews on CPU < 60 s	3.20 s	PASS
Maximum sequence length defended	L = 128 (Table 2)	PASS
Unseen-data Macro F1 ≥ 0.65 (extra check)	0.8116	PASS
English-pretrained (IMDB-style) initialisation compared	see Table 3	DONE (substitute source)
Human-labelled data; inter-annotator κ	none	NOT ACHIEVED

Selected model: LSTM, strategy A2: Scratch + robust, seed 42. Standard test: accuracy 0.9976, macro precision 0.9975, macro recall 0.9978, Macro F1 0.9976, weighted F1 0.9976, Mixed F1 0.9982, κ 0.9964. Unseen-data test: accuracy 0.8144, Macro F1 0.8116, Mixed F1 0.8514, κ 0.7217.

CPU benchmark. 10,000 distinct unseen reviews (no repeated test rows) were timed on CPU only (1 logical core, no GPU), including cleaning, normalisation, tokenisation, padding and prediction (batch 256), after two warm-up passes, as the median of three repeats: 3.20 s (2.59 s prediction only), 3,125 reviews/s; all six models took 1.9–3.3 s. The numeric targets are met on synthetic data only; this is not evidence of performance on real reviews.

5. ANSWERS TO THE REPORT QUESTIONS

5.1 Why did the simple RNN do worse on long reviews, and how do gates help?

Unrolled over T tokens, a simple recurrent network [1] is a T-layer graph with shared weights, $h_t = \tanh(W x_t + U h_{t-1} + b)$. The gradient to an early state is a product of T−t Jacobians $\mathrm{diag}(1 - h_k^2)\cdot U$; singular values below one make it vanish exponentially, above one make it explode [4], so an early cue barely influences the final decision and a late complaint can overwrite earlier praise in one state. LSTM adds a cell state $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ whose additive path, gated by forget, input and output gates, carries information and gradient over many steps [2]. GRU merges this into an update gate (interpolating old and candidate state) and a reset gate (how much past enters the candidate) [3].

Evidence. On the standard test, baseline SimpleRNN scored Macro F1 0.9855 versus 0.9988 (LSTM) and 0.9964 (GRU); at L = 64, where complaints are often truncated, validation Macro F1 was 0.903, 0.938 and 0.954 respectively. Results are not uniform: after robustness training GRU fell to 0.9493, below SimpleRNN (0.9763), and on unseen data the order was LSTM 0.8052 > SimpleRNN 0.7503 > GRU 0.7310. LSTM was consistently strong; the GRU advantage appeared only in some settings. With three seeds, seed SDs up to 0.044 and a synthetic corpus, this is consistent with the gating explanation for LSTM but does not prove it.

5.2 Did starting from the English IMDB model help, hurt, or make no difference?

Strategy B compared with strategy A (both without augmentation or normalisation); differences B − A, means over three seeds: SimpleRNN: accuracy -0.0004, Macro F1 -0.0001, Mixed F1 -

0.0036, unseen-data Macro F1 +0.0029; LSTM: accuracy +0.0008, Macro F1 +0.0008, Mixed F1 +0.0006, unseen-data Macro F1 -0.0068; GRU: accuracy +0.0032, Macro F1 +0.0032, Mixed F1 +0.0000, unseen-data Macro F1 -0.0131. Overall, English initialisation helped in some settings: changes larger than the seed spread are mostly positive.

Vocabulary overlap. Only 85 of 171 target tokens (50%) occur in the English source vocabulary, covering 35% of validation and test tokens. Frequent matched tokens include lo, story, review, opinion, theatre, waiting, weekend, ticket, normal, friends; frequent unmatched tokens include ., ga, undhi, ,, ki, undi, ee, naa, vyaktigata, kosam, which keep random embedding rows. The English source models were themselves weak (binary test accuracy SimpleRNN 0.468, LSTM 0.540, GRU 0.627), which limits what could transfer. Likely further reasons, not separately tested: the source is a small corpus of long English reviews (a different register from short code-mixed comments), English negation ("not good") differs from Telugu suffix negation ("bagaledu"), and the sentiment-bearing Romanized Telugu words have no English counterpart to transfer.

5.3 Error categories, and how much error comes from the model versus unclear labels

The selected model made 2 errors in 829 standard-test reviews: "Hero performance balledu" (true Negative, predicted Mixed; 3 tokens; spelling variant, negation); "Nagaram Nidra kosam chalaa waiting chesam, hero perf..." (true Negative, predicted Positive; 122 tokens; long review, spelling variant). On the unseen-data test it made 167 errors in 900. Table 5 groups errors by category (a review can carry several tags; – = category not generated or not tagged in that set).

Table 5. Errors by category, selected model (standard test vs unseen-data test)

Error category	Std test reviews	Std errors	Unseen reviews	Unseen errors
Sarcasm	85	0	–	–
Romanized spelling variation	648	2	64	1
Mixed opinions (true label Mixed)	276	0	300	45
Negation	325	1	–	–
Long-review context	284	1	278	58
Unseen vocabulary	0	0	836	166
Ambiguous label (hedge)	60	0	–	–
Telugu-English code mixing	829	2	900	167

Standard-test errors are too few to rank categories. On unseen data, reviews whose unseen tokens normalisation repaired (64 reviews) reached accuracy 0.984, whereas reviews containing a token it could not map (836) reached 0.801: a new sentiment word is read as an unknown token and the model must guess from context. Long reviews scored 0.791, Mixed reviews 0.850; the most frequent confusions were Positive → Negative (74) and Positive → Mixed (29). "Unseen word" includes heavily altered spellings the normaliser rejected, so it is not purely novel vocabulary.

Kappa. Cohen's $\kappa = (p_o - p_e)/(1 - p_e)$ [7] corrects observed agreement for chance. For the selected model $\kappa = 0.9964$ (standard) and 0.7217 (unseen data), measuring agreement between the model and the generator's rule labels. It is not inter-annotator agreement, which would need two or more humans labelling the same reviews and was not measured. Consequently the error cannot be split into model error and label ambiguity: the rule labels are deterministic, so by construction none of the error is label noise, and the hedged-ambiguous reviews (60 in the standard test, 0 errors) are ambiguous only by design. With real labels, a model–label κ well below the annotator–annotator κ would point to model error, and a similar value would point to unclear labels.

6. LIMITATIONS AND FUTURE WORK

(i) Synthetic, rule-labelled data with a small vocabulary and shared templates; the unseen-data sets come from the same authors, and their slang lexicon lacks native-speaker review. (ii) No human annotation, no inter-annotator κ, and no study of "very little labelled data" (6,000 reviews were used). (iii) The English transfer source was a small substitute corpus, not the Keras IMDB set, and the comparison uses three seeds. (iv) Hyper-parameters and bidirectional layers were not experimentally tested; the length sweep predates the robustness changes and used one seed. (v) CPU time was measured on a single-core sandbox for final models only. (vi) Similarity-based normalisation cannot recover new words and may map a word wrongly. Future work: collect and double-annotate a few thousand real reviews (Dravidian CodeMix-style), report annotator κ, repeat the transfer with the full Keras IMDB set in Colab, test bidirectional and character-level layers, and compare multilingual pretrained encoders such as multilingual BERT [10].

7. REFERENCES

[1] J. L. Elman, "Finding structure in time," Cognitive Science, vol. 14, no. 2, pp. 179–211, 1990.

[2] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[3] K. Cho et al., "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in Proc. EMNLP, 2014, pp. 1724–1734.

[4] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," IEEE Trans. Neural Netw., vol. 5, no. 2, pp. 157–166, 1994.

[5] B. Pang and L. Lee, "Opinion mining and sentiment analysis," Found. Trends Inf. Retr., vol. 2, no. 1–2, pp. 1–135, 2008.

[6] A. L. Maas et al., "Learning word vectors for sentiment analysis," in Proc. ACL-HLT, 2011, pp. 142–150.

[7] J. Cohen, "A coefficient of agreement for nominal scales," Educ. Psychol. Meas., vol. 20, no. 1, pp. 37–46, 1960.

[8] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. ICLR, 2015.

[9] S. S. V. Kusampudi, P. Sathineni, and R. Mamidi, "Sentiment analysis in code-mixed Telugu-English text with unsupervised data normalization." [Online]. Available: https://paperswithcode.com/paper/sentiment-analysis-in-code-mixed-telugu (venue and year to be confirmed by the team).

[10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171–4186.

[11] B. Pang and L. Lee, "A sentimental education: Sentiment analysis using subjectivity summarization based on minimum cuts," in Proc. ACL, 2004, pp. 271–278.

APPENDIX A – DECLARATION OF AI TOOL USE

An AI assistant (Claude, by Anthropic) was used to inspect the earlier notebook, rewrite the data generator, preprocessing, experiments, robustness techniques and evaluation code, execute the notebook, and draft this report and the documentation. Figures were read by script from the executed notebook's output files. The team must independently review the code, results and references, and confirm each reference's details against its source; no such verification is claimed here. Every member must be able to explain the code line by line in the viva.